

The Compiler

Rust, In Theory and in Practice

CAS CS 392 M1

The Point

The Point

We consider Rust from the perspective of **interpreters** because it helps us build **mental models**

The Point

We consider Rust from the perspective of **interpreters** because it helps us build **mental models**

But Rust is ultimately a **compiled** language, because it wants to be **efficient**

Questions of the Day

Questions of the Day

What's going on in the compiler? Maybe even,
what is a compiler?

Questions of the Day

What's going on in the compiler? Maybe even, what is a compiler?

When do things like **borrow checking** or **pattern exhaustiveness** or **monomorphization** happen?

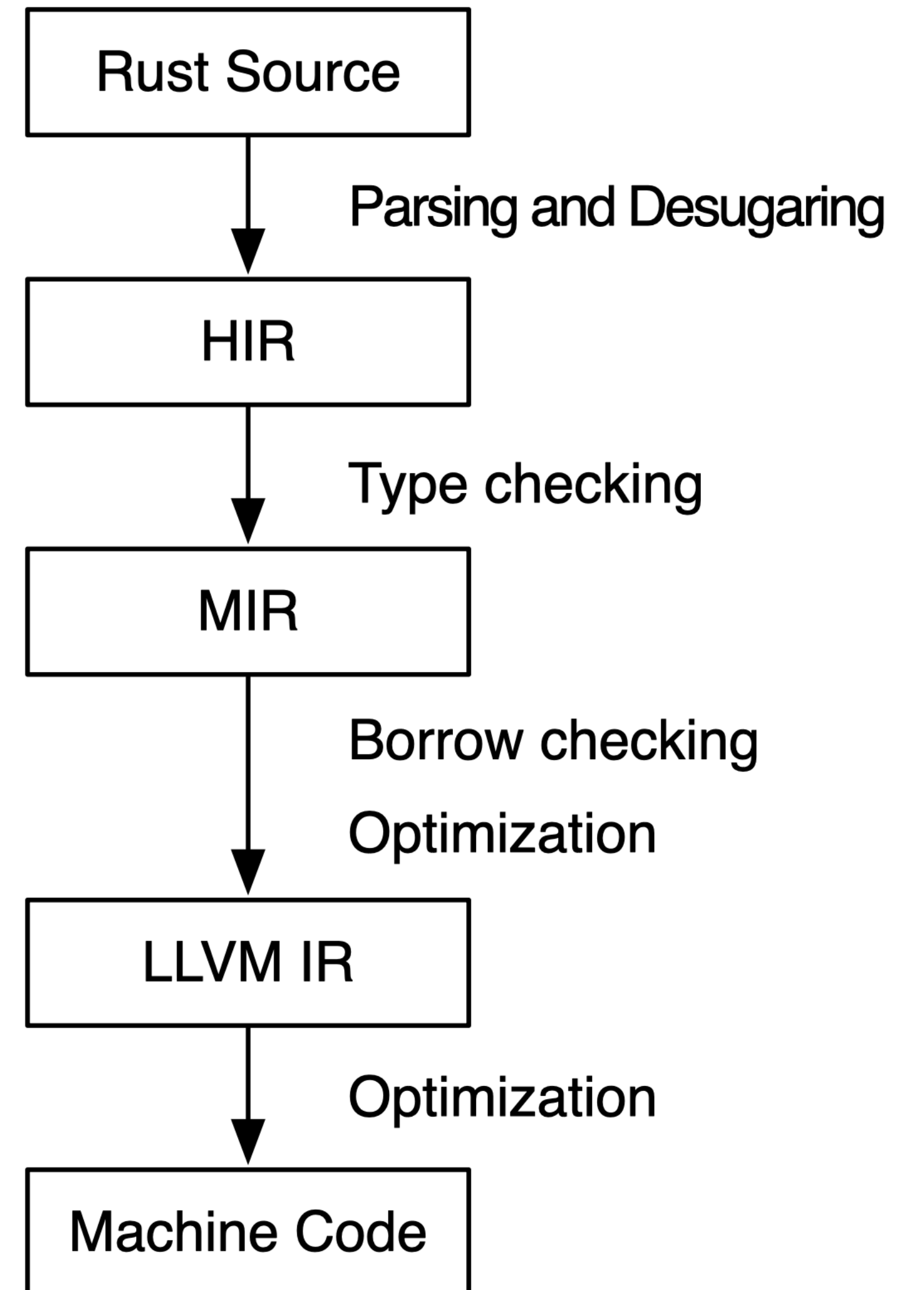
Questions of the Day

What's going on in the compiler? Maybe even, what is a compiler?

When do things like **borrow checking** or **pattern exhaustiveness** or **monomorphization** happen?

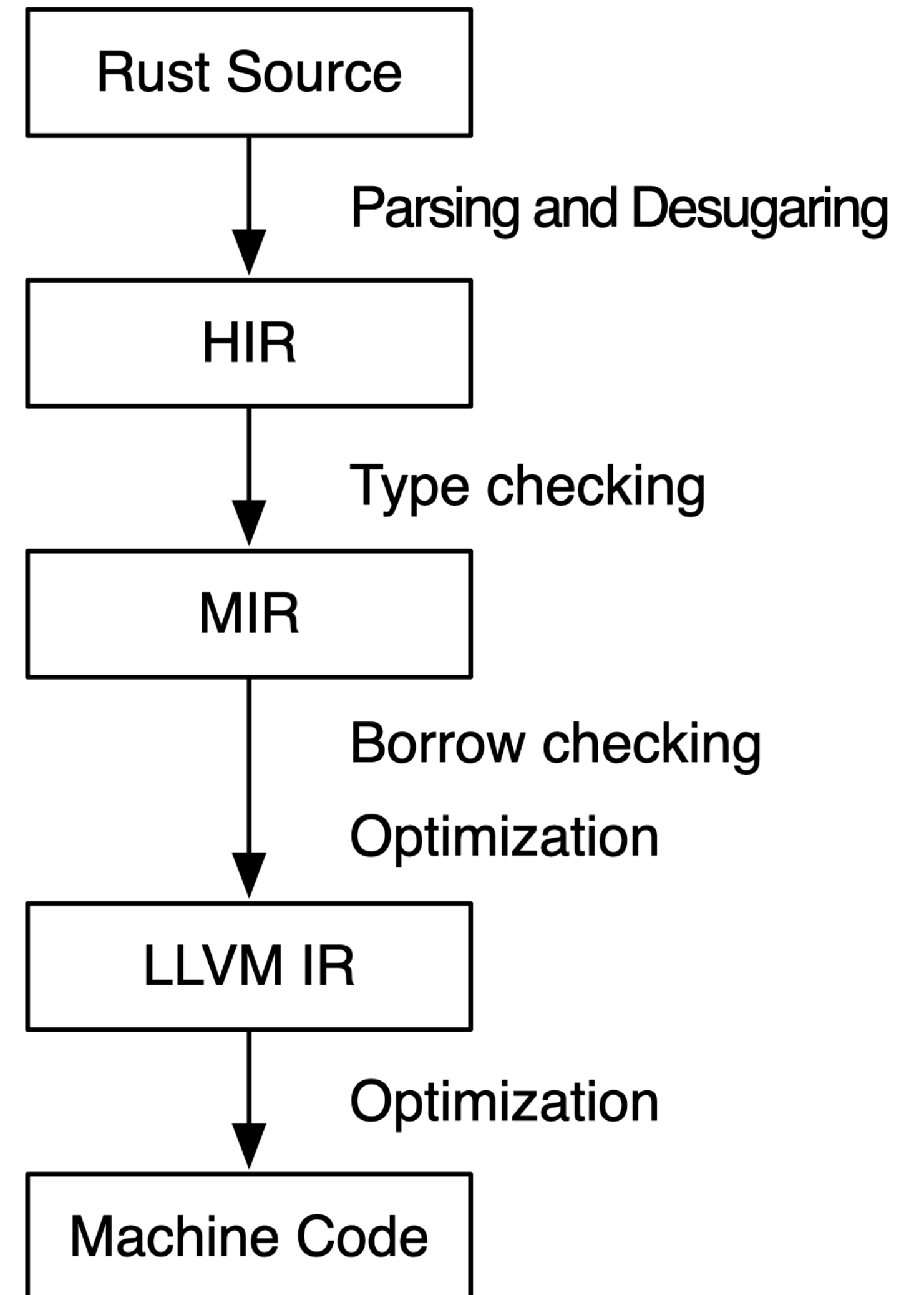
How does this relate to our interpreter view?

Compilers



Compilers

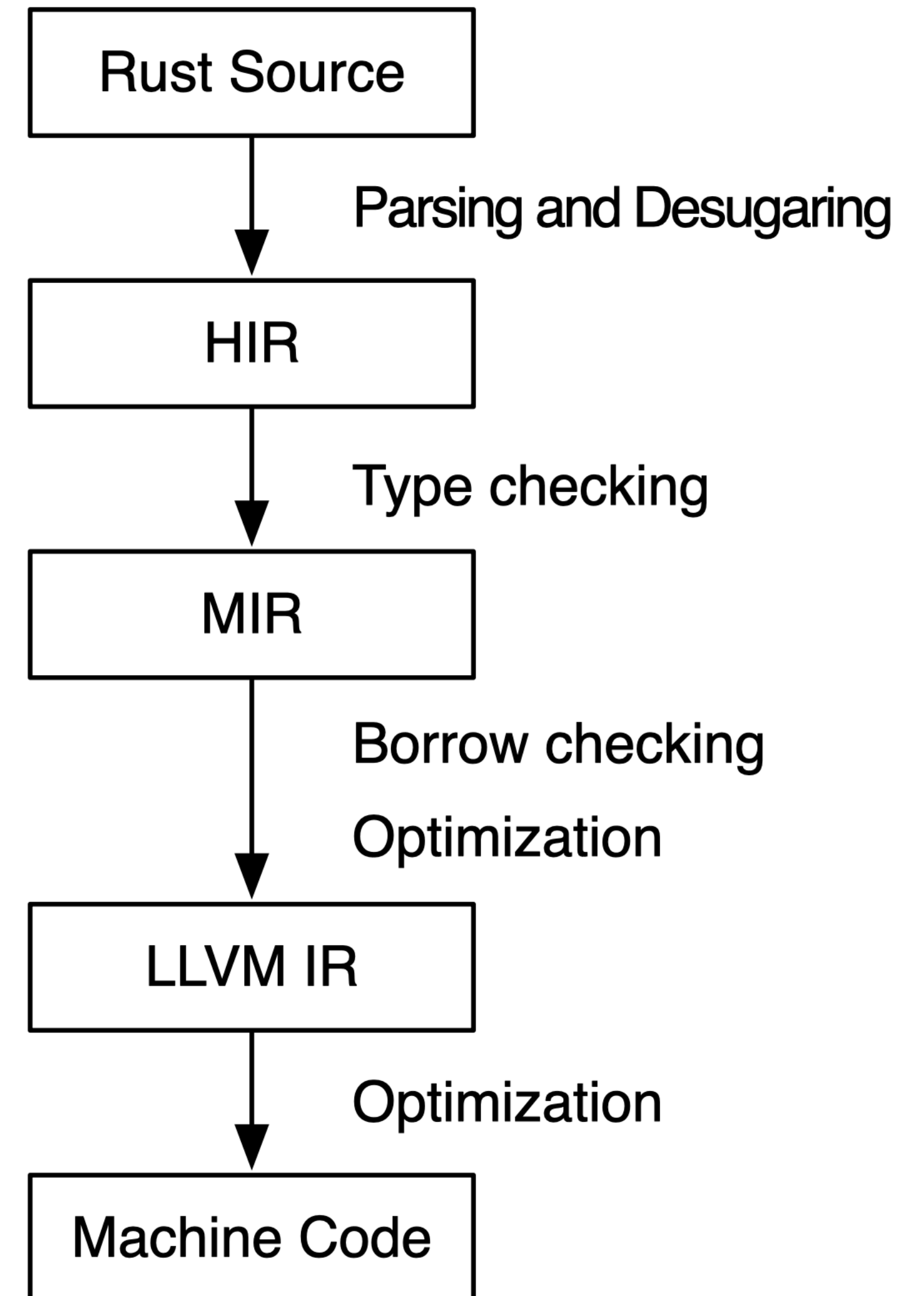
A compiler (often) consists of a sequence of *lowerings* to different **intermediate representations (IR)**



Compilers

A compiler (often) consists of a sequence of *lowerings* to different **intermediate representations (IR)**

An IR may be simpler or easier to analyze or more informative

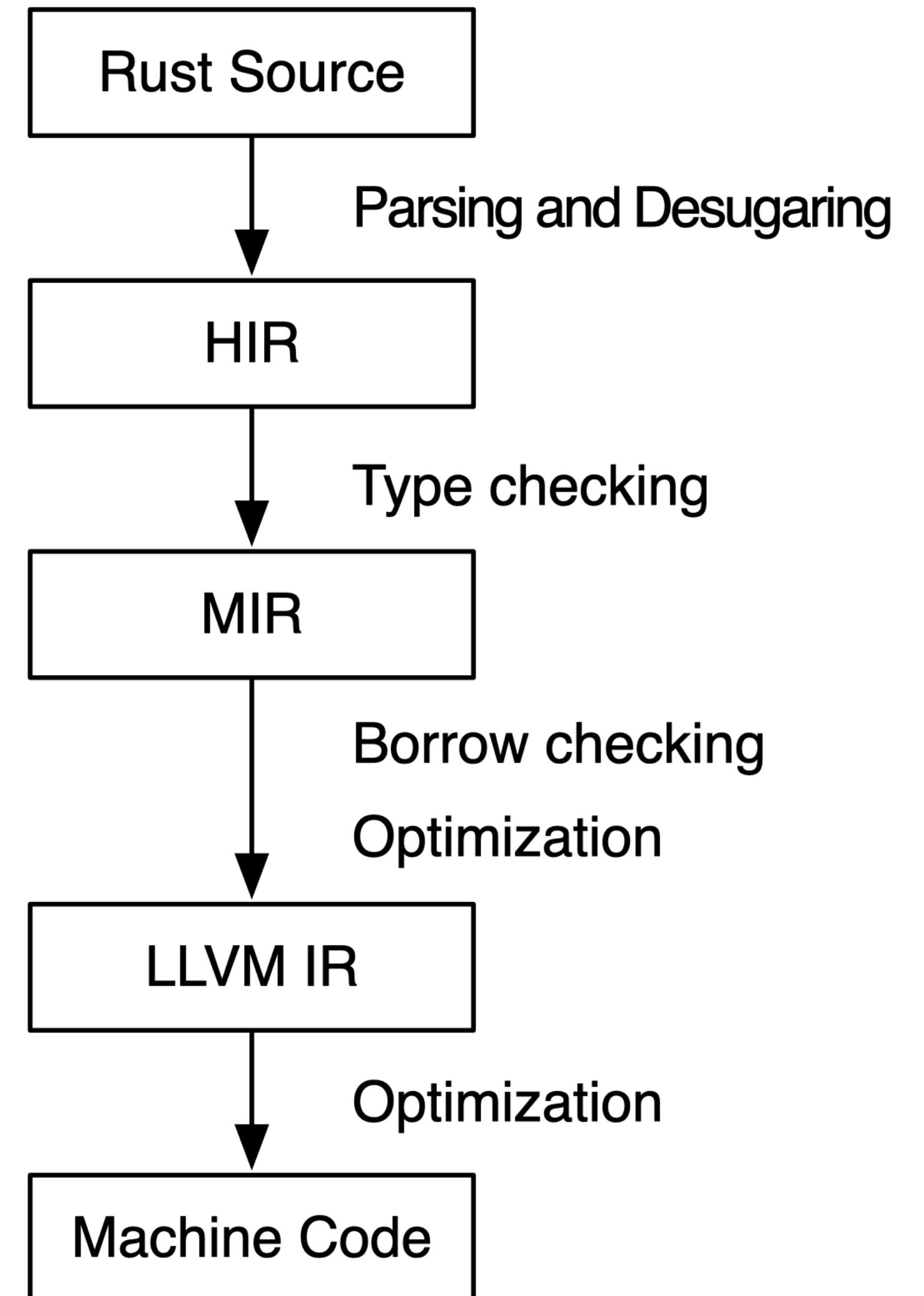


Compilers

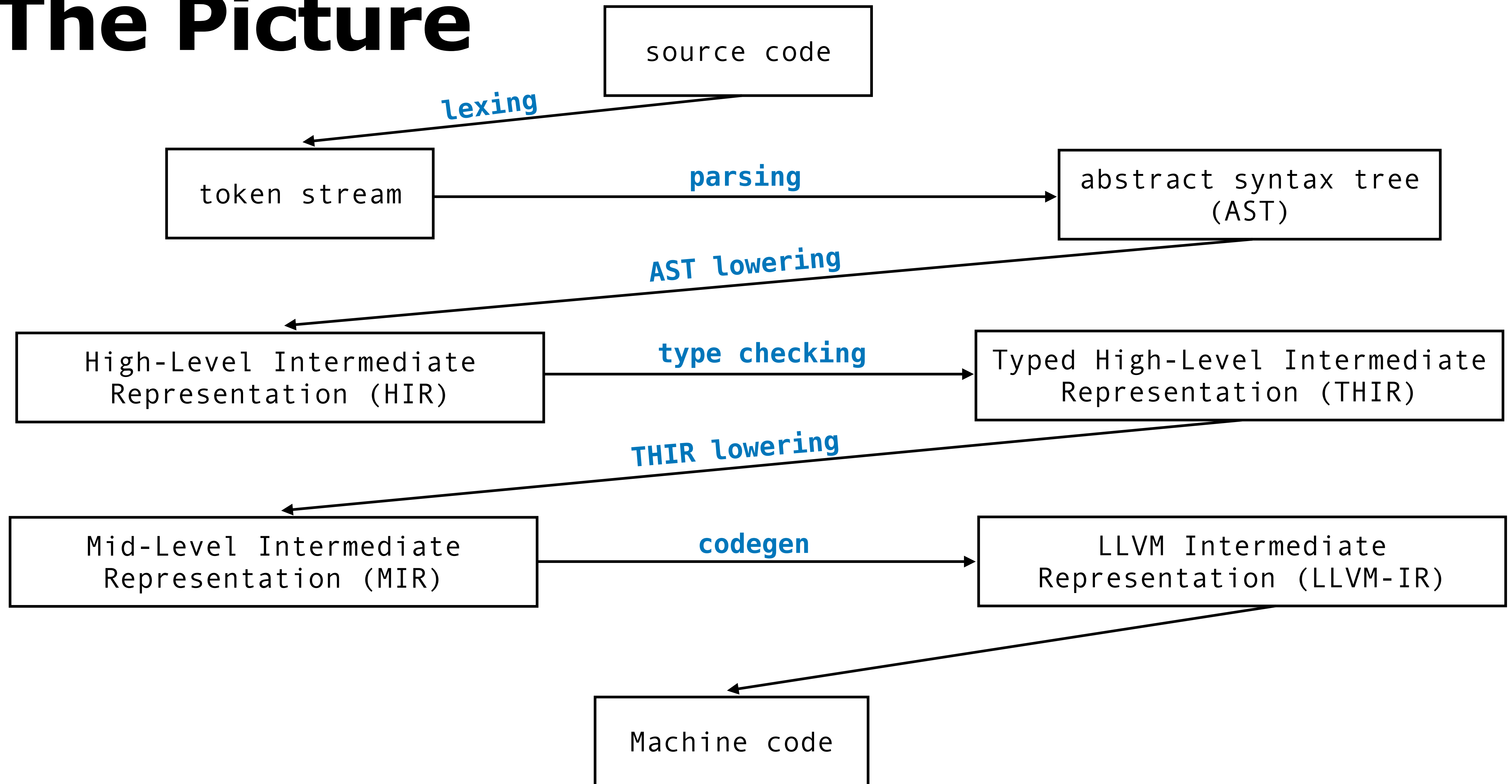
A compiler (often) consists of a sequence of *lowerings* to different **intermediate representations (IR)**

An IR may be simpler or easier to analyze or more informative

Each transformation must maintain the *semantics* of the original source code

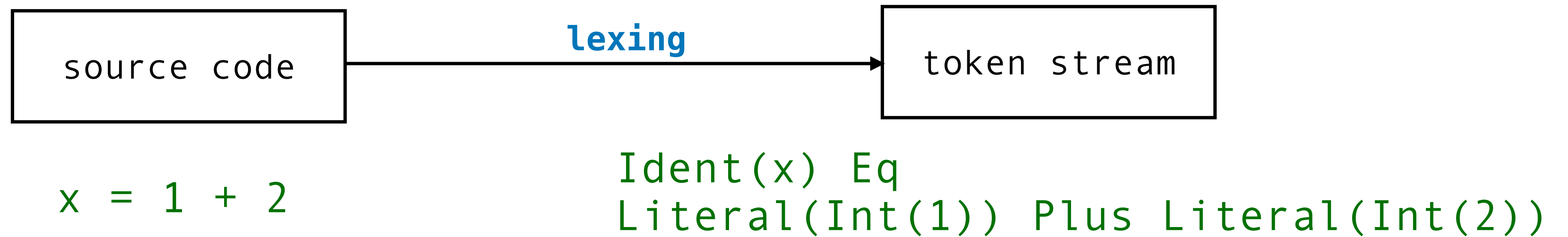


The Picture



Syntactic Analysis

Lexing



Lexing is the process of turning a character string into a *token* stream

e.g., in salt and Rust

Aside: String Interning

Compilation is a *performance sensitive* endeavor

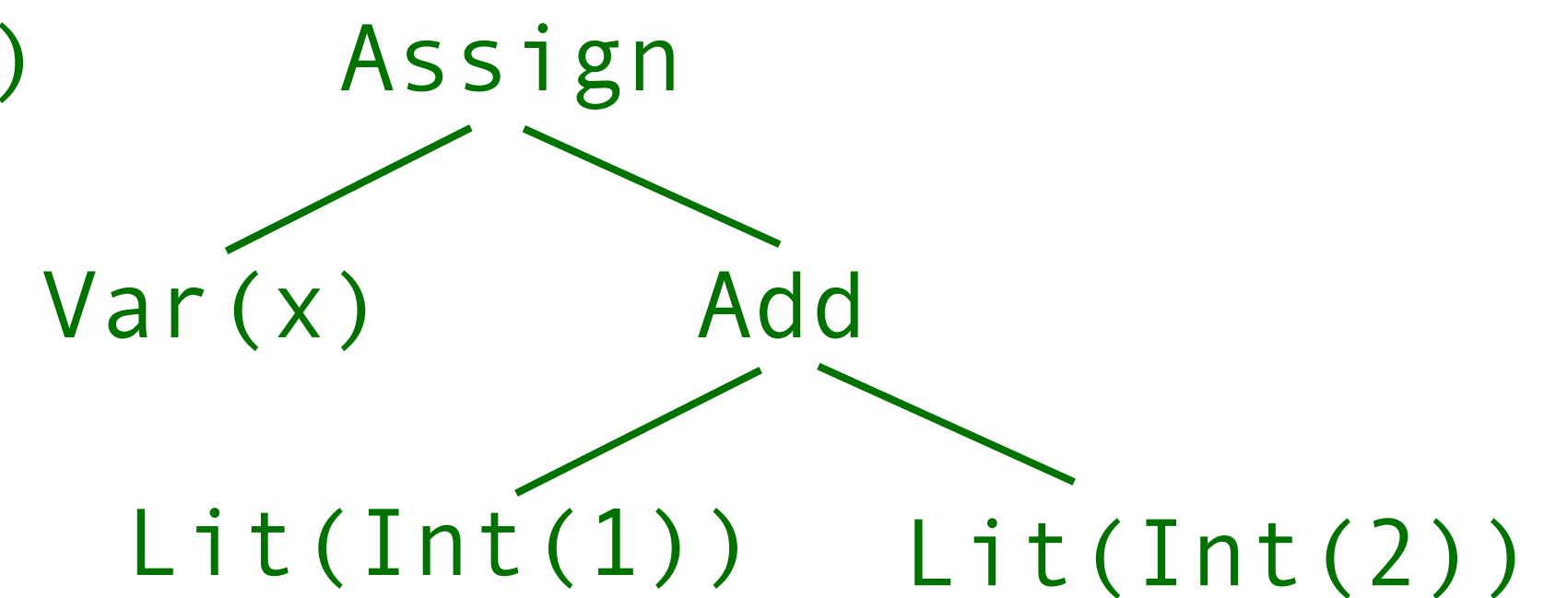
Things like variable names are **immutable** from the perspective of the compiler, so we can use this fact represent variable names as *pointers* to a fixed string value

e.g. in Rust

Parsing



`Ident(x) Eq Literal(Int(1)) Plus Literal(Int(2))`



Parsing give our token stream hierarchical structure, and gives us an *abstract syntax tree*

This is also where **macros** and **attributes** are dealt with

e.g., in salt and Rust

High-Level IR

Aside: Emitting

```
rustc +nightly -Z unpretty=??? file.rs
```

We can emit the IRs using the unpretty flag with options:

<code>`normal`,</code>	<code>`identified`,</code>	<code>`expanded`,</code>
<code>`expanded,identified`,</code>	<code>`expanded,hygiene`,</code>	<code>`ast-tree`,</code>
<code>`ast tree,expanded`,</code>	<code>`hir`,</code>	<code>`hir,identified`,</code>
<code>`hir,typed`,</code>	<code>`hir-tree`,</code>	<code>`thir-tree`,</code>
<code>`thir-flat`,</code>	<code>`mir`,</code>	<code>`stable-mir`,</code>
<code>`mir-cfg`</code>		

What happens?



Parentheses are removed (they are a part of the AST for pretty-printing)

Things like **for-loops** and **async** are expanded

impl Traits are made into generics

e.g., in Rust

Typed High-Level IR

What happens?



The HIR is type checked and everything is annotated with typing information

Some additional structure is made concrete (implicit derefs are made explicit)

e.g., in Rust

Mid-Level IR

What happens?



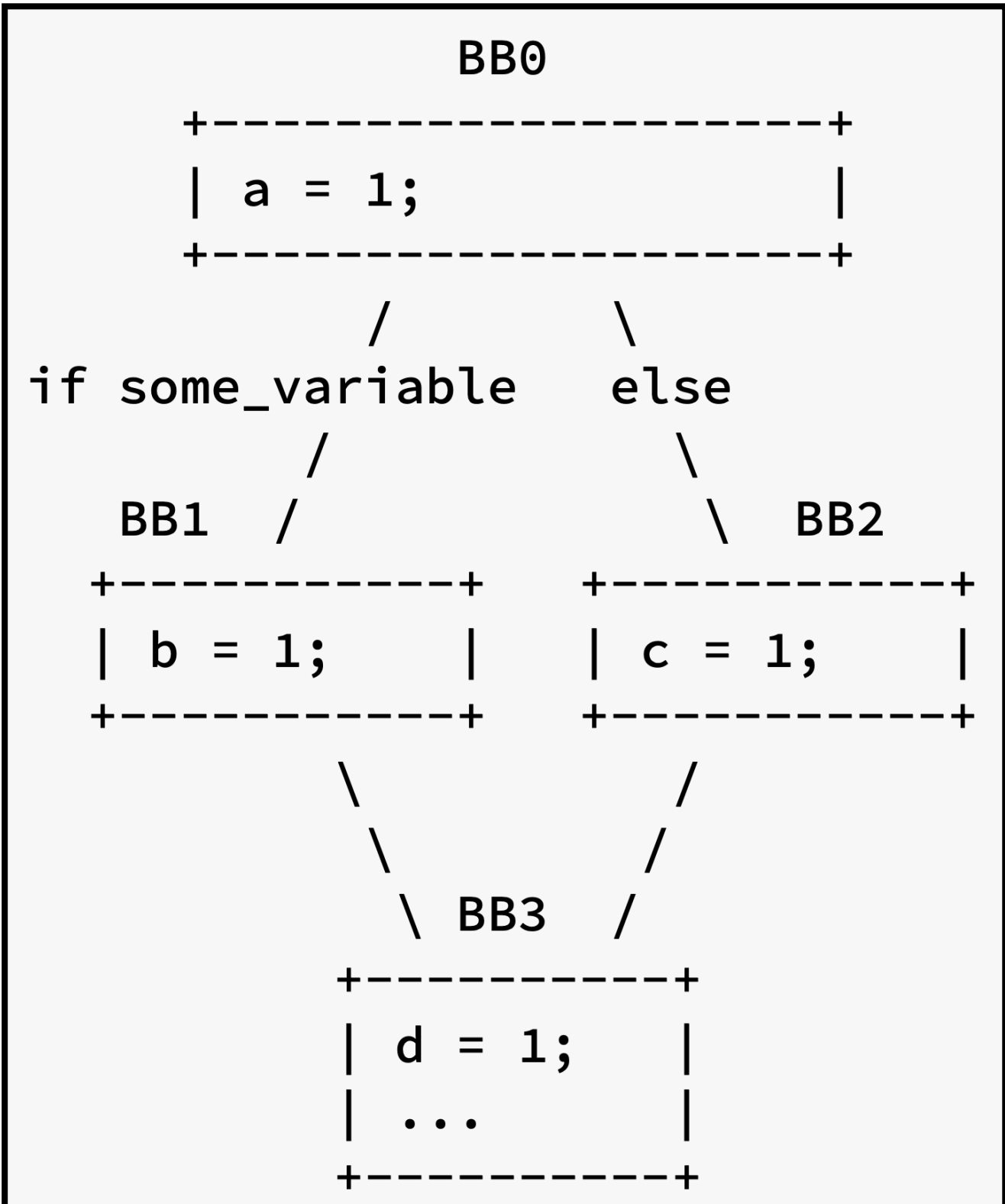
Blocks/function bodies are recursively "flattened" into a sequence of statements, connected by GOTOs

This is essentially a **control flow graph** which can be used for analyzing data usage/computation at execution time

e.g., in Rust

Control Flow Graph

```
BB0: {  
    a = 1;  
    if some_variable {  
        goto BB1;  
    } else {  
        goto BB2;  
    }  
}  
  
BB1: {  
    b = 1;  
    goto BB3;  
}  
  
BB2: {  
    c = 1;  
    goto BB3;  
}  
  
BB3: {  
    d = 1;  
    ...  
}
```



A **control flow graph** is a DAG which maintains information about what computational paths might be taken during execution

Analyzing the MIR

Rust uses several techniques from **static analysis** to analyze the MIR of your program:

- » Dataflow analysis is used to check for initialization, borrows, and moves
- » Constraint propagation for lifetimes and region inference

And Beyond

What happens?



MIR is borrow checked, optimized, and then is lowered to LLVM-IR, doing monomorphization along the way

This is passed to LLVM for further optimization and generation of machine code

The Picture (Again)

